

Studia.

Organização da rotina acadêmica — matérias, atividades com prazo, avaliações e progresso, em um único lugar e 100% offline.

Documentação do Projeto

INSTITUIÇÃO	Universidade Estadual de Montes Claros — Cept
CURSO	Técnico em Desenvolvimento de Sistemas
DISCIPLINA	Desenvolvimento de Sistemas para Dispositivos Móveis
INTEGRANTES	Pedro Miguel · Anna Cecilia
TURMA	Turma 1
PROFESSOR(A)	Isa Alexandre
DATA	09/10/2026

00 Sumário

Este documento reúne, conforme a Etapa 1 do roteiro, todas as decisões de planejamento, arquitetura e implementação do Studia. Ele descreve o que foi projetado **e o que foi efetivamente construído** — o código no repositório é a fonte de verdade executável.

1	Introdução	8	Protótipo das Telas
2	Objetivo	9	Modelagem do Banco de Dados
3	Problema	10	Diagrama do Banco (ER)
4	Público-alvo	11	Dados Utilizados
5	Requisitos Funcionais	12	Tecnologias
6	Requisitos Não Funcionais	13	Organização do Código
7	Telas e Fluxo de Navegação	14	Conclusão

Identificação do projeto

Código-fonte versionado em github.com/heyalmeida/studia-app, com acesso de avaliação concedido à professora. Este documento reúne a Etapa 1 (planejamento e documentação) e reflete o que foi efetivamente implementado no repositório.

1 Introdução

O **Studia** é um aplicativo móvel de organização de estudos, desenvolvido em dupla como Projeto Final da disciplina de Desenvolvimento de Sistemas para Dispositivos Móveis.

O app permite ao estudante centralizar a rotina acadêmica em um único lugar: cadastrar **matérias**, registrar **atividades** (tarefas, trabalhos, leituras e estudos) com prazo e situação, acompanhar **avaliações** (provas, seminários e entregas) por data e visualizar, na tela inicial, um resumo do que vence primeiro, do que falta por matéria e do progresso geral.

Foi construído com **React Native + Expo + TypeScript** sobre o projeto-base da disciplina, seguindo a stack exigida. O aplicativo é **100% local e offline**: não há servidor, conta de usuário ou dependência de internet — os dados são persistidos no próprio aparelho via AsyncStorage.

Além do produto em si, o projeto foi conduzido com **Desenvolvimento Orientado a Especificação (Spec-Driven Development)**: cada decisão de arquitetura e funcionalidade está registrada em documentação versionada junto ao código (requisitos, especificações por feature, Architecture Decision Records e planos de execução), o que torna o repositório legível como portfólio técnico.

2 Objetivo

Demonstrar, na prática, os conteúdos da disciplina — organização de código, navegação entre telas, componentes reutilizáveis, formulários com validação, listas e manipulação de dados — por meio de um aplicativo que resolve um problema real do estudante.

Objetivos do aplicativo (o que o usuário deve conseguir fazer)

- Cadastrar e manter **matérias**;
- Registrar **atividades** vinculadas a uma matéria, com prazo, tipo e situação;
- Marcar atividades como **concluídas**, enxergando o progresso por matéria;
- Registrar **avaliações** (provas) com data, vinculadas a uma matéria;
- Abrir o app e ver, na **tela inicial**, um resumo da rotina: pendências, prazos próximos e provas futuras;
- **Buscar e filtrar** por matéria nas listas e receber um **lembrete local** um dia antes de um prazo.

Proposta de valor

Um único lugar onde o estudante registra o que tem por fazer (por matéria), enxerga o que vence primeiro e acompanha o quanto já concluiu. O Studia não tenta ser agenda completa nem plataforma de aprendizagem — ele responde, em segundos, a três perguntas: *o que vence primeiro? o que falta por matéria? como andam minhas provas?*

3 Problema

Estudantes gerenciam a rotina acadêmica de forma **fragmentada**: prazos anotados no caderno, provas esquecidas até a véspera, materiais espalhados entre aplicativos que não se conversam.

O resultado concreto dessa fragmentação é triplo: **perda de prazos, estudo reativo** (estudar "quando dá", sem planejamento) e **ausência de visão objetiva** do que ainda falta fazer por matéria. Não existe uma ferramenta simples que una o registro do que fazer, o acompanhamento do prazo e a noção de progresso — sem exigir internet, cadastro ou configuração complexa.

O Studia ataca exatamente esse ponto: ele reúne registro, prazo e progresso em uma única tela inicial, funciona no bolso durante o dia todo (offline) e não pede nada além de um cadastro rápido de matéria para começar a ser útil.

Hipótese registrada no planejamento: o problema é plausível e comum, embora não tenha sido validado com usuários reais — suficiente para o escopo de um projeto disciplinar.

4 Público-alvo

Estudantes de cursos técnicos e de graduação que cursam **várias matérias simultâneas** e precisam organizar tarefas, trabalhos e provas em um único lugar.

Cenário típico de uso

- Uso no **celular**, em curtas sessões de consulta e registro ao longo do dia;
- Sem exigência de internet — o app funciona no ônibus, no intervalo, onde estiver;
- Perfil que hoje se vira com caderno, planilha ou mais de um app que não conversam entre si.

O aplicativo é autoexplicativo e não possui contas nem múltiplos usuários: cada pessoa usa o próprio aparelho com seus próprios dados. Isso mantém a barreira de entrada baixa — o requisito central é que o estudante consiga registrar e consultar sem atrito.

5 Requisitos Funcionais

O roteiro exige no mínimo 6 requisitos funcionais; o Studia possui **9**, derivados naturalmente do domínio de três entidades (nenhum requisito foi criado apenas para atingir o mínimo).

ID	Requisito	Descrição e critérios de aceitação
RF-01	Cadastrar matéria	Informa nome (obrigatório, ≥2 caracteres, único sem diferenciar maiúsculas) e professor(a) (opcional). Nome vazio ou duplicado bloqueia o cadastro com mensagem específica.
RF-02	Listar e visualizar matérias	Cards ordenados por nome, exibindo por matéria: atividades pendentes, avaliações agendadas e progresso (concluídas ÷ total). Toque no card abre a edição.
RF-03	Editar e excluir matéria	Edição abre preenchida e reaplica as validações. Exclusão exige confirmação e é bloqueada quando há atividades/avaliações vinculadas ("exclua-os primeiro").
RF-04	Cadastrar atividade	Atividade (tarefa, trabalho, leitura ou estudo) com título, matéria (obrigatória), prazo (opcional) e descrição. Data no passado gera aviso , não bloqueio. Sem matérias, o formulário orienta a cadastrar uma antes.
RF-05	Listar e filtrar atividades	Ordenadas por prazo (mais próximos primeiro, sem prazo ao final), com filtro por situação: todas, pendentes, concluídas. Cada item mostra matéria, prazo destacado e situação.
RF-06	Concluir / reabrir atividade	Alternar a situação diretamente na lista, atualizando o progresso da matéria e o resumo do painel, com feedback imediato e sem recarregar a tela.
RF-07	Editar e excluir atividade	Correção com as mesmas validações do cadastro; exclusão com confirmação, removendo a atividade de listas, painel e agregados.
RF-08	Cadastrar e gerenciar avaliações	Avaliação com título, matéria e data obrigatória ; lista ordenada pela data (agendadas primeiro); alternância entre agendada e realizada. O histórico de realizadas permanece visível.
RF-09	Painel inicial de visão da rotina	Tela de abertura com resumo: pendências, prazos próximos, próximas avaliações e progresso geral. Cada bloco navega para a tela relacionada; com dados zerados, exibe estado vazio convidativo.

Requisitos funcionais adicionais (expansões aprovadas)

Ref.	Requisito	Descrição
RF-10	Busca e filtro por matéria	Busca textual (sem diferenciar maiúsculas nem acentos) e filtro por matéria nas listas de atividades, avaliações e matérias, sem alterar a ordenação.

Ref.	Requisito	Descrição
RF-11	Lembrete local de prazo	Opt-in por registro: agenda uma notificação local 1 dia antes do prazo, às 08:00. Concluir, excluir ou desligar o lembrete cancela o agendamento.

6 Requisitos Não Funcionais

O roteiro exige no mínimo 4; o Studia define **6**, cada um ligado a um aspecto real do produto e com forma de verificação objetiva.

ID	Requisito	Descrição e verificação
RNF-01	Usabilidade e interface organizada	No máximo 2 toques entre abrir o app e uma ação comum; alvos de toque $\geq 44\text{pt}$; padrão visual consistente via UI kit compartilhado; cor nunca usada sozinha (sempre com rótulo textual). Verificação: inspeção por tela + percurso "abrir $\rightarrow$ ver o que vence hoje".
RNF-02	Organização do código em camadas	Pastas por responsabilidade (<code>app</code> , <code>screens</code> , <code>components</code> , <code>domain</code> , <code>storage</code> , <code>services</code> , <code>hooks</code> ...), sem lógica concentrada em um arquivo; ≥ 3 componentes reutilizáveis em mais de uma tela. Verificação: revisão da árvore <code>src/</code> .
RNF-03	Manutenibilidade e segurança de tipos	TypeScript <code>strict</code> , sem <code>any</code> gratuito, ESLint do Expo sem erros, commits pequenos e claros. Verificação: <code>tsc --noEmit</code> e <code>expo lint</code> limpos.
RNF-04	Tratamento de erros e feedback	Mensagens claras por campo, falha de armazenamento sem texto técnico, confirmação em exclusão, feedback de sucesso; o app não quebra diante de dado ausente ou corrompido. Verificação: roteiro manual por tela.
RNF-05	Compatibilidade mobile (Android prioritário)	Orientação retrato, respeito a safe areas, layout fluido, tema escuro único com contraste verificado. Verificação: dispositivo físico/emulador Android no checklist da Etapa 6.
RNF-06	Persistência confiável e offline	Todo dado sobrevive a fechar e reabrir o app; funciona integralmente sem rede; gravação atômica por coleção. Verificação: ciclo cadastrar $\rightarrow$ fechar $\rightarrow$ reabrir + teste em modo avião.

7 Telas e Fluxo de Navegação

O roteiro exige no mínimo 5 telas navegáveis; o Studia possui **7**. As quatro primeiras são **abas**; as três de formulário são **rotas modais** sobre a pilha. Cada tela de listagem concentra o próprio CRUD (listar + editar/excluir), atendendo ao alvo de " ≤ 2 toques para ação comum".

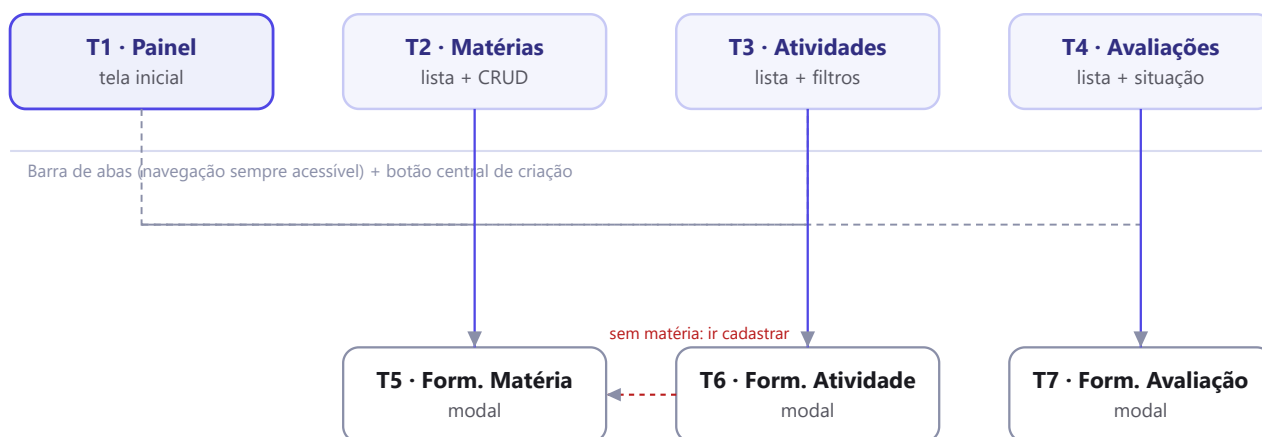


Figura 1 — Fluxo de navegação: abas (T1–T4), formulários modais (T5–T7) e a ponte de T6 para T5 quando não há matéria cadastrada.

Descrição das telas

Tela	Objetivo	Informações e ações
T1	Painel (inicial)	Saudação + data; card principal com anel de progresso, "X de Y concluídas" e quantas vencem em 7 dias; "Próximos prazos" (3 atividades) e próxima avaliação. Toque abre edição ou "Ver todas".
T2	Matérias	Cards com ícone tingido na cor da matéria, nome, professor, contadores e barra de progresso na cor. Busca por nome/professor. Botão de criação abre T5; toque no card edita.
T3	Atividades	Controle segmentado (pendentes/todas/concluídas), agrupamento por período (Atrasadas, Hoje, Esta semana, Depois), item com título, matéria e chip de prazo por urgência. Busca + filtro por matéria. Conclusão por checkbox.
T4	Avaliações	Cards com bloco de data, título, matéria e contagem regressiva; realizadas esmaecidas como histórico. Busca + filtro por matéria; alternar agendada/realizada.
T5	Form. Matéria	Nome, professor(a), carga horária, grade de ícones e seletor de cor (8 tons). Rodapé fixo com Salvar / Cancelar / Excluir (edição).
T6	Form. Atividade	Título, seletor de matéria, tipo com ícone, prazo com calendário + atalhos, descrição e switch "Lembrar". Ponte para T5 quando não há matéria.
T7	Form. Avaliação	Título, seletor de matéria, data obrigatória com calendário e switch "Lembrar". Mesmo padrão de T6, sem tipo/descrição.

8 Protótipo das Telas

O roteiro exige o protótipo de, no mínimo, 5 telas. As capturas abaixo correspondem às telas finais do aplicativo (desenvolvidas em Figma / telas reais) e devem ser inseridas nos quadros reservados, uma por tela.

T1 · Painel

inserir protótipo

T2 · Matérias

inserir protótipo

T3 · Atividades

inserir protótipo

T5 · Form. Matéria

inserir protótipo

T6 · Form. Atividade

inserir protótipo

T4 · Avaliações

inserir protótipo (opcional)

Cinco quadros atendem ao mínimo exigido; o sexto é opcional. Recomenda-se versionar as imagens em [docs/prototype/](#) junto ao repositório (questão OQ-08).

9 Modelagem do Banco de Dados

A modelagem cobre **3 entidades relacionadas** (mínimo do roteiro). Embora o app não use banco relacional — a persistência é local por chave-JSON (AsyncStorage) —, campos, tipos, chaves e relacionamentos estão formalizados abaixo.

Subject (Matéria)

Campo	Tipo	Chave	Finalidade
id	string (UUID)	PK	Identificador estável entre coleções
name	string	—	Nome exibido; obrigatório, ≥2 caracteres, único
teacher	string ou nulo	—	Professor(a); opcional
hour	número ou nulo	—	Horas por semana; opcional
icon	string ou nulo	—	Nome do ícone (grade); opcional
color	string ou nulo	—	Tom da paleta de 8; nulo = sem cor
createdAt	string (ISO 8601)	—	Auditoria

Activity (Atividade)

Campo	Tipo	Chave	Finalidade
id	string (UUID)	PK	—
subjectId	string	FK → Subject.id	Matéria da atividade (obrigatória)
title	string	—	O que deve ser feito; obrigatório
type	tarefa trabalho leitura estudo	—	Classificação leve
description	string ou nulo	—	Detalhamento opcional
dueDate	string (só-data) ou nulo	—	Prazo; ordena a lista e alimenta o painel
status	pendente concluída	—	Situação; motor do progresso
completedAt	string (ISO) ou nulo	—	Data da conclusão
reminder	boolean	—	Lembrete local ligado (1 dia antes, 08:00)
notificationId	string ou nulo	—	Id do agendamento; nulo = nada agendado
createdAt	string (ISO 8601)	—	Auditoria

Assessment (Avaliação)

Campo	Tipo	Chave	Finalidade
id	string (UUID)	PK	—
subjectId	string	FK → Subject.id	Matéria da prova (obrigatória)
title	string	—	Identificação da avaliação; obrigatório
date	string (só-data)	—	Data; obrigatória — o que a diferencia de Activity
status	agendada realizada	—	Situação; histórico permanece visível
reminder	boolean	—	Lembrete local ligado (1 dia antes, 08:00)
notificationId	string ou nulo	—	Id do agendamento; nulo = nada agendado
createdAt	string (ISO 8601)	—	Auditoria

Relacionamentos e cardinalidade

Relacionamento	Cardinalidade	Leitura e totalidade
Subject → Activity	1 : N	Uma matéria possui muitas atividades; cada atividade pertence a exatamente uma matéria (FK obrigatória).
Subject → Assessment	1 : N	Uma matéria possui muitas avaliações; cada avaliação pertence a exatamente uma matéria (FK obrigatória).

Atividade e Avaliação **não** se relacionam entre si. A integridade da chave estrangeira não é imposta por banco (não há banco relacional), e sim pela regra de domínio: é impossível criar atividade ou avaliação sem matéria válida pela interface, e excluir uma matéria com registros vinculados é bloqueado. Registros órfãos são tratados como dado corrompido: as agregações ignoram chaves inexistentes em vez de quebrar a tela.

10 Diagrama do Banco (ER)

Diagrama entidade-relacionamento simplificado das três entidades, com atributos, chaves primárias (**PK**), chaves estrangeiras (**FK**) e a cardinalidade 1:N entre Matéria e suas atividades/avaliações.

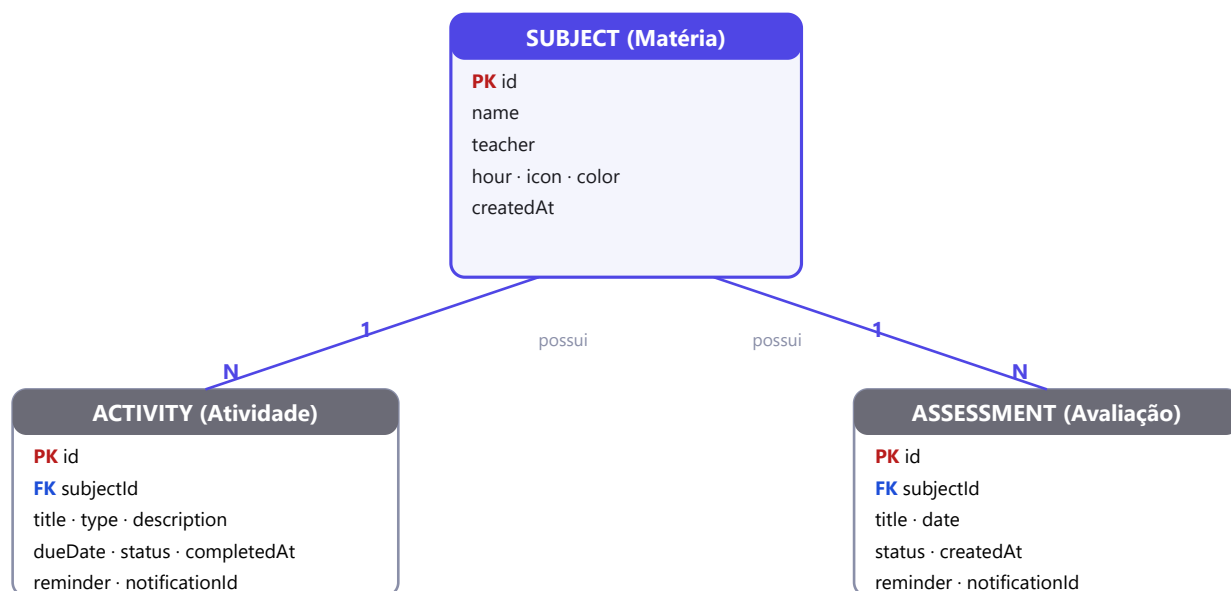


Figura 2 — Diagrama ER: uma matéria para muitas atividades e muitas avaliações (1:N).

Progresso é derivado, não armazenado

$\text{progresso}(\text{matéria}) = \text{atividades concluídas} \div \text{atividades totais}$ é **computado** a partir das entidades, nunca persistido — evitando inconsistência entre a fonte e o derivado. Não existe campo de nota: o progresso do Studia é este (decisão OQ-04).

11 Dados Utilizados

O Studia adota a **Opção 2 — Armazenamento Local** do roteiro. Não há API externa nem back-end: o próprio aparelho é a fonte de dados, o que garante funcionamento 100% offline.

Fluxo dos dados

Usuário cadastra → **o app salva** (AsyncStorage, uma chave por coleção) → **o app recupera** ao abrir → **listas e telas exibem**.

Coleções e telas onde aparecem

Chave (storage)	Entidade	Cadastrada / consultada em
<code>studia.subjects</code>	Matérias	Cadastro/edição em T5; consulta em T2, T1 e nos seletores de T6/T7
<code>studia.activities</code>	Atividades	Cadastro/edição em T6; consulta em T3, T1 e agregados de T2
<code>studia.assessments</code>	Avaliações	Cadastro/edição em T7; consulta em T4, T1 e agregados de T2

Como os dados chegam às telas

- A UI **nunca** acessa o storage diretamente: consome **hooks** (`use-subjects`, `use-activities`, `use-assessments`, `use-dashboard`) que leem os **repositórios** por trás de uma interface.
- Uma camada de **pub/sub** (notifier) avisa quando uma coleção muda; assim, concluir uma atividade atualiza o card da matéria e o painel sem recarregar a tela.
- A leitura é **defensiva**: JSON ausente ou inválido vira lista vazia; campos opcionais de versões anteriores são migrados na leitura (registros antigos abrem sem erro).

12 Tecnologias

Stack alinhada ao projeto-base da disciplina (React Native + Expo + TypeScript), com o acréscimo pontual de bibliotecas para calendário, ícones, gráficos e notificação local.

Framework de app	React Native 0.86 sobre Expo SDK 57
Linguagem	TypeScript 6 (modo <code>strict</code>)
Biblioteca de UI	React 19
Navegação	Expo Router 57 (abas + rotas modais)
Estilização	<code>StyleSheet</code> do React Native + tokens centralizados
Persistência	@react-native-async-storage/async-storage
Ícones	lucide-react-native
Gráficos	react-native-svg (anel de progresso)
Notificação local	expo-notifications (agendamento local)
Segurança de tipos / qualidade	ESLint (eslint-config-expo) + tsc
Versionamento	Git + GitHub

Decisão de estilo relevante (portfólio)

O projeto adotou inicialmente NativeWind (Tailwind) e migrou para `StyleSheet` puro: o motor do NativeWind é um módulo nativo que **não existe dentro do Expo Go**, onde o app é executado e demonstrado. A decisão está registrada em ADR. Isso mantém o app fiel ao requisito do roteiro de "organizar o layout com StyleSheet e Flexbox".

13 Organização do Código

O código é organizado em camadas por responsabilidade, sem concentrar lógica em um único arquivo. As rotas ficam em `src/app/` (layouts + re-exports finos); cada tela é uma pasta em `src/screens/`.

Estrutura de pastas

Pasta	Responsabilidade
<code>src/app/</code>	Rotas Expo Router — layouts de navegação e re-exports finos das telas
<code>src/screens/</code>	Implementação das telas (uma pasta <code><Tela>Page/</code> por tela)
<code>src/components/</code>	UI: <code>ui/</code> = kit reutilizável multi-tela; <code><Tela>Page/</code> = componentes de uma tela
<code>src/hooks/</code>	Hooks de dados que compõem repositórios e reagindo ao notifier
<code>src/domain/</code>	Entidades, regras de negócio e validações puras (sem React nem storage)
<code>src/storage/</code>	Persistência — repositórios sobre AsyncStorage (único ponto que conhece o storage)
<code>src/services/</code>	Integrações de sistema (expo-notifications — lembretes locais)
<code>src/constants/</code>	Fonte única dos tokens visuais (cores, tipografia, espaçamentos)
<code>assets/</code>	Imagens e ícones do aplicativo

Componentes reutilizáveis (mínimo do roteiro: 3)

O kit em `src/components/ui/` reúne cerca de **26 componentes** usados em mais de uma tela, entre eles:

- `ScreenHeader` (cabeçalho com título e ações), `Card`, `ListItem`, `Divider`;
- `Button` / `PrimaryButton`, `TextButton`, `CreateButton`, `Touchable`;
- `Input`, `FormField`, `SearchField`, `Chip`, `ChoiceChip`, `SubjectChip`, `SubjectFilterRow`;
- `IconTile`, `IconPicker`, `ColorPicker`, `Monogram`;
- `DateField`, `DatePicker`, `DateBlock`, `DueChip`;
- `ProgressBar`, `ProgressRing`, `EmptyState`, `ListSkeleton`, `SegmentedControl`.

A regra de ouro da arquitetura: a camada de UI consome apenas hooks; os hooks consomem apenas repositórios por interface; e o domínio é feito de funções puras, testáveis isoladamente.

14 Conclusão

O Studia foi entregue como um aplicativo móvel completo e funcional, cumprindo — e superando — os requisitos mínimos do roteiro.

Durante o projeto, a dupla desenvolveu um app com **7 telas navegáveis** (mínimo 5), **9 requisitos funcionais** mais 2 expansões (mínimo 6), **6 requisitos não funcionais** (mínimo 4), **3 entidades relacionadas** com chaves e cardinalidade documentadas (mínimo 3) e um **UI kit com cerca de 26 componentes reutilizáveis** (mínimo 3). O aplicativo implementa formulários com validação real, listas ordenadas e filtráveis, busca sem acentos, lembrete local de prazo e persistência offline que sobrevive a fechar e reabrir o app.

Além do produto, o projeto exercitou práticas de engenharia de software aplicadas ao desenvolvimento móvel: arquitetura em camadas com domínio puro, gerenciamento de estado reativo por pub/sub, segurança de tipos com TypeScript estrito, tratamento defensivo de dados corrompidos e um fluxo de trabalho guiado por especificação e decisões registradas (ADRs). O histórico de commits no repositório demonstra a evolução do projeto etapa por etapa.

Como aprendizado central, ficou a percepção de que **decisões de plataforma moldam a arquitetura**: a escolha de manter o app executável no Expo Go para a demonstração levou, por exemplo, a migrar o sistema de estilo para [StyleSheet](#) e a preferir notificação local a push remoto. O Studia demonstra que é possível entregar, com ferramentas estudadas na disciplina, uma solução real, organizada e pronta para uso.

Repositório do código-fonte: github.com/heyalmeida/studia-app · Acesso de avaliação concedido à professora. Este documento acompanha a entrega por e-mail conforme a Etapa 7 do roteiro.